

Chapter 2: Operating-System Structures

Operating System Concepts, 7th Edition — Silberschatz, Galvin, Gagne

1. Operating System Services

Operating system services fall into two broad groups: those helpful to the user, and those that ensure the efficient operation of the system itself.

1.1 Services Helpful to the User

- **User interface (UI)** — almost all operating systems provide one; this can be Command-Line (CLI), Graphical (GUI), or Batch
- **Program execution** — the system must be able to load a program into memory, run it, and end its execution, either normally or abnormally (indicating an error)
- **I/O operations** — a running program may require I/O involving a file or an I/O device
- **File-system manipulation** — programs need to read and write files and directories, create and delete them, search them, list file information, and manage permissions
- **Communications** — processes may exchange information, on the same computer or over a network, either via shared memory or through message passing (with the OS moving packets)
- **Error detection** — the OS must be constantly aware of possible errors, which may occur in the CPU and memory hardware, in I/O devices, or in the user program; for each type of error, the OS should take appropriate action to ensure correct and consistent computing. Debugging facilities can greatly enhance users' and programmers' ability to use the system efficiently

1.2 Services for Efficient System Operation (Resource Sharing)

- **Resource allocation** — when multiple users or jobs run concurrently, resources must be allocated among them. Some resources (CPU cycles, main memory, file storage) may need special allocation code, while others (I/O devices) may use general request-and-release code
- **Accounting** — keeping track of which users use how much, and what kinds, of computer resources
- **Protection and security** — owners of information in a multiuser or networked system may want to control its use, and concurrent processes should not interfere with each other. Protection means ensuring all access to system resources is controlled; security means defending the system from outsiders, requiring user authentication and extending to defending external I/O devices from invalid access. As the notes put it: a chain is only as strong as its weakest link, so precautions must be instituted throughout the system

2. System Calls

System calls provide the programming interface to the services provided by the OS. They are typically written in a high-level language such as C or C++.

Programs mostly access these services through a high-level Application Program Interface (API) rather than invoking system calls directly. The three most common APIs are the Win32 API (Windows), the POSIX API (used by virtually all versions of UNIX, Linux, and Mac OS X), and the Java API (for the Java Virtual Machine, JVM). Using an API instead of a raw system call gives portability across systems that implement the same API, and can be more convenient than working with the underlying system call directly. (System-call names used throughout the textbook are generic.)

2.1 System Call Implementation

- Typically, a number is associated with each system call, and the system-call interface maintains a table indexed by these numbers
- The system call interface invokes the intended system call in the OS kernel, and returns its status along with any return values
- The caller needs to know nothing about how the system call is implemented — it just needs to obey the API and understand what the OS will do as a result of the call
- Most details of the OS interface are hidden from the programmer by the API, and managed by the run-time support library — a set of functions built into the libraries included with a compiler

2.2 Types of System Calls

- Process control
- File management
- Device management
- Information maintenance
- Communications

3. Operating System Design and Implementation

Designing and implementing an OS is not a fully "solvable" problem, but some approaches have proven successful. The internal structure of different operating systems can vary widely. Design begins by defining goals and specifications, and is affected by the choice of hardware and the type of system.

Design goals fall into two categories:

- User goals — the OS should be convenient to use, easy to learn, reliable, safe, and fast
- System goals — the OS should be easy to design, implement, and maintain, as well as flexible, reliable, error-free, and efficient

3.1 Policy vs. Mechanism

An important design principle is to separate policy from mechanism:

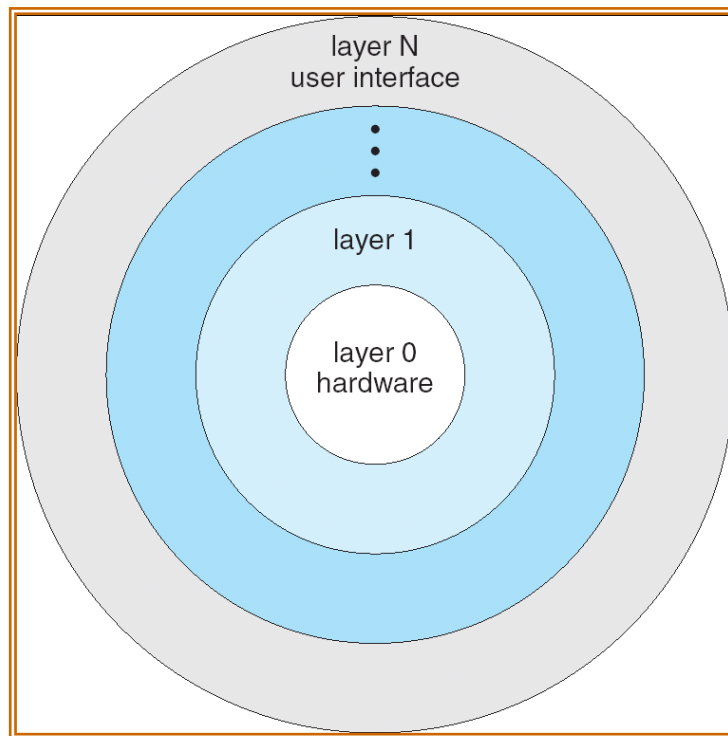
- Policy — what will be done?
- Mechanism — how is it to be done?

Mechanisms determine how to do something; policies decide what will be done. This separation is an important principle because it allows maximum flexibility if policy decisions need to be changed later, without needing to alter the underlying mechanism.

4. Operating System Structure

4.1 Layered Approach

The operating system is divided into a number of layers (levels), each built on top of the layers below it. The bottom layer (layer 0) is the hardware; the highest layer (layer N) is the user interface. With modularity, each layer is designed so that it uses only the functions (operations) and services of lower-level layers.

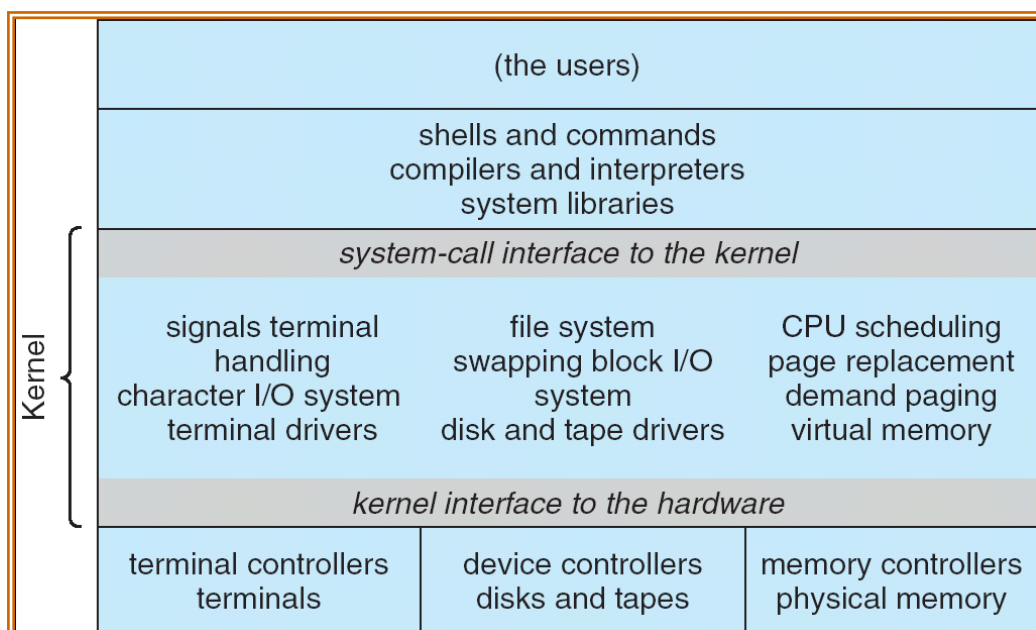


4.2 UNIX System Structure

Limited by the hardware functionality of its time, the original UNIX operating system had limited structuring. The UNIX OS consists of two separable parts:

- System programs
- The kernel — consisting of everything below the system-call interface and above the physical hardware. It provides the file system, CPU scheduling, memory management, and other operating-system functions — a large number of functions bundled into a single level

In the UNIX layering (from the hardware upward): hardware (terminal, device, and memory controllers) → kernel interface to the hardware → kernel functions (signals, terminal handling, character I/O, terminal drivers; file system, swapping, block I/O, disk/tape drivers; CPU scheduling, page replacement, demand paging, virtual memory) → system-call interface to the kernel → shells and commands, compilers and interpreters, system libraries → the users.



5. Virtual Machines

A virtual machine (VM) takes the layered approach to its logical conclusion. It treats the hardware and the operating-system kernel as though they were all hardware, and provides an interface identical to the underlying bare hardware. The operating system creates the illusion of multiple processes, each executing on its own processor with its own (virtual) memory.

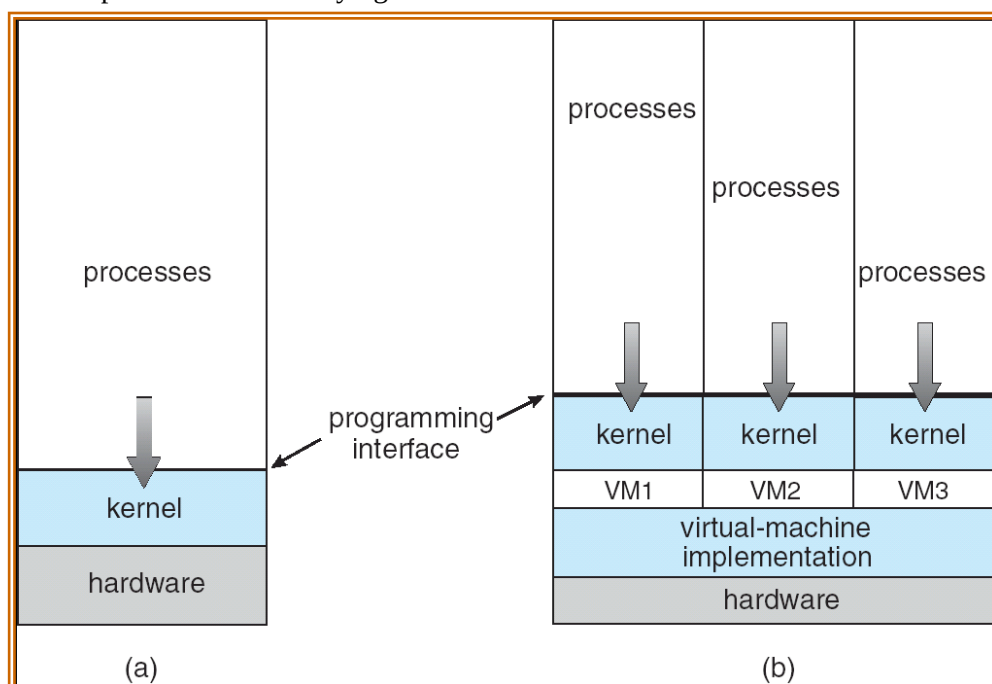
The resources of the physical computer are shared to create the virtual machines:

- CPU scheduling can create the appearance that each user has their own processor
- Spooling and a file system can provide virtual card readers and virtual line printers
- A normal user time-sharing terminal serves as the virtual machine operator's console

In a non-virtual machine, processes run directly on a kernel that sits on the hardware. In a virtual-machine system, several independent virtual machines (each with its own kernel and processes) run on top of a virtual-machine implementation layer, which itself runs on the hardware.

Advantages and limitations:

- Complete protection of system resources, since each virtual machine is fully isolated from all other virtual machines — but this isolation permits no direct sharing of resources
- A virtual-machine system is a perfect vehicle for operating-systems research and development, since development can be done on the virtual machine instead of a physical machine, without disrupting normal system operation
- The virtual machine concept is difficult to implement, due to the effort required to provide an exact duplicate of the underlying machine



6. System Boot

The operating system must be made available to the hardware so that the hardware can start it.

- A small piece of code called the bootstrap loader locates the kernel, loads it into memory, and starts it
- This is sometimes a two-step process, where a boot block at a fixed location loads the bootstrap loader

- When the system's power is initialized, execution starts at a fixed memory location; firmware is used to hold this initial boot code

Multiple Choice Questions (10)

1. Which of the following is an OS service primarily helpful to the user, rather than for efficient system operation?

- a) Resource allocation
- b) Accounting
- c) Program execution
- d) Protection and security

Answer: (c) Program execution

2. System calls provide:

- a) A hardware interrupt mechanism only
- b) The programming interface to the services provided by the OS
- c) A user interface for file management only
- d) A layered approach to memory management

Answer: (b) The programming interface to the services provided by the OS

3. Which of these is NOT one of the three most common APIs mentioned for accessing OS services?

- a) Win32 API
- b) POSIX API
- c) Java API
- d) SQL API

Answer: (d) SQL API

4. In system call implementation, the system-call interface uses a table indexed by:

- a) Process IDs
- b) A number associated with each system call
- c) File descriptors
- d) Memory addresses

Answer: (b) A number associated with each system call

5. Which of the following is NOT one of the five types of system calls listed?

- a) Process control
- b) File management
- c) Device management
- d) Network topology configuration

Answer: (d) Network topology configuration

6. In operating system design, 'Policy' refers to:

- a) How something will be done
- b) What will be done
- c) The hardware layer
- d) The user interface only

Answer: (b) What will be done

7. In the layered approach to OS structure, which layer is the hardware?

- a) Layer N
- b) The middle layer
- c) Layer 0
- d) There is no fixed layer for hardware

Answer: (c) Layer 0

8. In the original UNIX system structure, everything below the system-call interface and above the physical hardware is called the:

- a) Shell
- b) Kernel
- c) API
- d) Bootstrap loader

Answer: (b) Kernel

9. A key limitation of the virtual machine concept, resulting from its complete isolation between virtual machines, is that it:

- a) Cannot run more than one process
- b) Permits no direct sharing of resources between virtual machines
- c) Cannot be used for OS research and development
- d) Requires no hardware resources at all

Answer: (b) Permits no direct sharing of resources between virtual machines

10. The small piece of code that locates the kernel, loads it into memory, and starts it during system boot is called the:

- a) Dispatcher
- b) Bootstrap loader
- c) System-call interface
- d) Run-time support library

Answer: (b) Bootstrap loader

Five-Mark Questions (5)

1. Describe the operating system services that are helpful to the user and those that exist for ensuring the efficient operation of the system itself. Give at least three examples in each category with a brief explanation.
2. What are system calls? Explain how a system call is typically implemented, and discuss why programs are usually written to use a high-level API (such as Win32, POSIX, or Java API) rather than invoking system calls directly.
3. Describe the layered approach to operating system structure, and illustrate how UNIX's original system structure reflects (or departs from) this approach, using the roles of the kernel and system programs.
4. What is a virtual machine? Explain how the resources of a physical computer are shared to create virtual machines, and discuss the advantages and difficulties associated with implementing the virtual machine concept.